



The University of Azad Jammu and Kashmir

Name: Zarnab Fatima

Roll NO: 2024-SE-14

Semester:4th

Course Title: Database System

Course Code: CS-2204

Submitted To: Engr. Awais Rathore

Department of Software Engineering

LAB 12

Aggregate Functions in SQL

1. Objective

The main objectives of this lab are:

- To understand aggregate functions in SQL.
- To use "COUNT()", "SUM()", "AVG()", "MIN()", and "MAX()" functions.
- To understand the difference between aggregate and scalar functions.
- To use "GROUP BY" for grouping records.
- To understand the use of "HAVING" for filtering groups.
- To handle "NULL" values correctly in aggregate queries.
- To perform aggregate calculations using multiple tables and joins.
- To calculate totals, averages, minimum values, maximum values, and counts from database records.

2. Introduction

Aggregate functions are SQL functions that perform calculations on a group of rows and return a single result. They are commonly used to summarize and analyze data stored in database tables.

The most commonly used aggregate functions are:

- "COUNT()" – Counts the number of rows.
- "SUM()" – Calculates the total of numeric values.
- "AVG()" – Calculates the average value.
- "MIN()" – Finds the minimum value.
- "MAX()" – Finds the maximum value.

Aggregate functions are often used with "GROUP BY" when we need separate results for different categories or groups.

3. Database Creation

First, the database named "agg_lab" is created.

SQL Query

```
CREATE DATABASE IF NOT EXISTS agg_lab;
```

```
USE agg_lab;
```

Explanation

This query creates a database named "agg_lab" if it does not already exist. The "USE" command selects the database so that all tables can be created inside it.



4. Creating and Insert Tables

A screenshot of a database management tool's table list. The table has columns: Table, Action, Rows, Type, Collation, Size, and Overhead. It lists three tables: customer, orderitem, and product. Each table has a set of actions (Browse, Structure, Search, Insert, Empty, Drop) and a summary row for all three tables.

Table	Action	Rows	Type	Collation	Size	Overhead
☐ customer		0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ orderitem		0	InnoDB	utf8mb4_general_ci	48.0 KiB	-
☐ product		0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
3 tables	Sum	0	InnoDB	utf8mb4_general_ci	80.0 KiB	0 B

☐ Check all With selected:

SELECT * FROM `customer`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Sort by key:

Extra options

				CustID	CustName	City	JoinDate
☐				1	Ali Khan	Lahore	2022-01-15
☐				2	Sara Iqbal	Karachi	2022-04-22
☐				3	Hamza Raza	Lahore	2023-02-10
☐				4	Ayesha Noor	Islamabad	2023-05-18
☐				5	Bilal Ahmed	Karachi	2023-09-01
☐				6	Fatima Sheikh	NULL	2024-01-12
☐				7	Usman Tariq	Lahore	2024-06-30
☐				8	Maira Javed	Islamabad	2024-08-25

☐ Check all With selected: Edit Copy Delete Export

SELECT * FROM `orderitem`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Sort by key:

Extra options

				OrderID	CustID	ProdID	Quantity	OrderDate
☐				1001	1	101	1	2023-03-10
☐				1002	1	102	2	2023-03-10
☐				1003	2	104	1	2023-05-22
☐				1004	2	106	5	2023-05-22
☐				1005	3	101	1	2023-08-15
☐				1006	3	110	1	2023-08-15
☐				1007	4	108	3	2023-11-02
☐				1008	5	103	4	2024-01-20
☐				1009	5	102	1	2024-01-20
☐				1010	6	105	1	2024-02-14
☐				1011	7	107	2	2024-04-08
☐				1012	7	106	10	2024-04-08
☐				1013	7	109	3	2024-07-19

SELECT * FROM `product`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				ProdID	ProdName	Category	Price	StockQty
☐	Edit	Copy	Delete	101	Laptop Pro 15	Electronics	185000.00	12
☐	Edit	Copy	Delete	102	Wireless Mouse	Electronics	2500.00	50
☐	Edit	Copy	Delete	103	USB-C Cable	Electronics	800.00	100
☐	Edit	Copy	Delete	104	Office Chair	Furniture	18500.00	8
☐	Edit	Copy	Delete	105	Standing Desk	Furniture	45000.50	5
☐	Edit	Copy	Delete	106	Notebook A4	Stationery	350.00	200
☐	Edit	Copy	Delete	107	Ballpoint Pen 10pk	Stationery	450.00	150
☐	Edit	Copy	Delete	108	Coffee Beans 1kg	Grocery	1899.99	30
☐	Edit	Copy	Delete	109	Green Tea Box	Grocery	650.00	45
☐	Edit	Copy	Delete	110	Bluetooth Speaker	Electronics	7500.00	18

☐ Check all | With selected: Edit Copy Delete Export

PART A — BASIC AGGREGATE FUNCTIONS

A1. Count Customers, Products and Orders

Objective

To count the total number of customers, products, and orders.

SQL Query

SELECT

(SELECT COUNT(*) FROM Customer) AS TotalCustomers,

(SELECT COUNT(*) FROM Product) AS TotalProducts,

(SELECT COUNT(*) FROM OrderItem) AS TotalOrders;

Explanation

The "COUNT()" function counts the total number of records in the three tables.

```
-- Task A1 SELECT (SELECT COUNT(*) FROM Customer) AS TotalCustomers, (SELECT COUNT(*) FROM Product) AS TotalProducts, (SELECT COUNT(*) FROM OrderItem) AS TotalOrders;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

TotalCustomers	TotalProducts	TotalOrders
8	10	15

A2. Find Minimum and Maximum Product Price

Objective

To find the lowest and highest product prices.

SQL Query

SELECT

MIN(Price) AS MinPrice,

MAX(Price) AS MaxPrice

FROM Product;

Explanation

"MIN()" returns the lowest product price, while "MAX()" returns the highest product p.

```
-- Task A2 SELECT MIN(Price) AS MinPrice, MAX(Price) AS MaxPrice FROM Product;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 ▼ | Filter rows:

Extra options

MinPrice	MaxPrice
350.00	185000.00

A3. Calculate Average Product Price

Objective

To calculate the average price of all products.

SQL Query

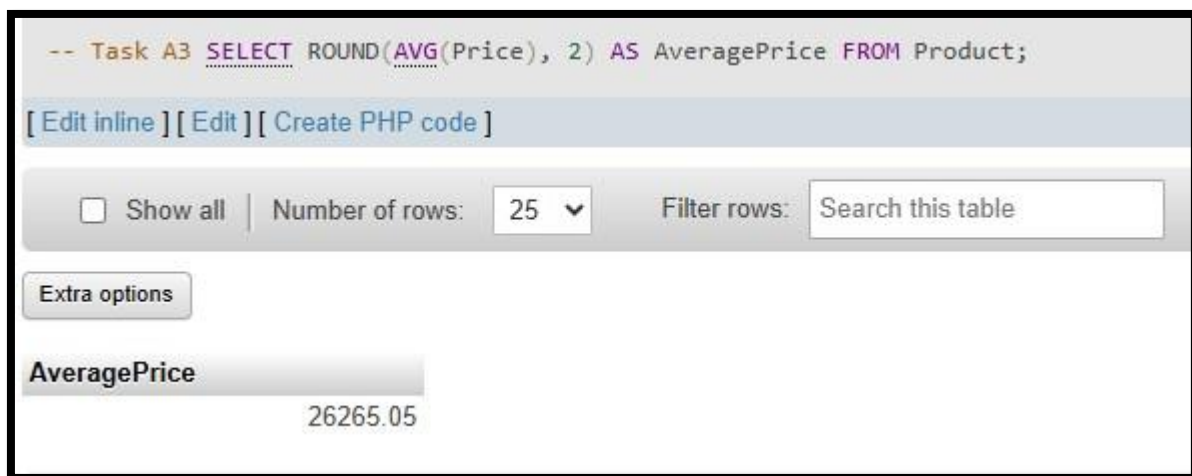
SELECT

ROUND(AVG(Price), 2) AS AveragePrice

FROM Product;

Explanation

"AVG()" calculates the average product price. "ROUND()" displays the result up to two decimal places.



A4. Calculate Total Stock

Objective

To calculate the total quantity of products available in stock.

SQL Query

SELECT

SUM(StockQty) AS TotalStockQuantity FROM

Product;

Explanation

The "SUM()" function adds the stock quantity of all products.

```
-- Task A4 SELECT SUM(StockQty) AS TotalStockQuantity FROM Product;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 ▼ | Filter rows:

Extra options

TotalStockQuantity
618

A5. Count Distinct Customer Cities

Objective

To find the number of different cities of customers.

SQL Query

```
SELECT
    COUNT(DISTINCT City) AS DistinctCities
FROM Customer
WHERE City IS NOT NULL;
```

Explanation

"DISTINCT" removes duplicate cities. The "WHERE" condition excludes "NULL" city values.

```
-- Task A5 SELECT COUNT(DISTINCT City) AS DistinctCities FROM Customer WHERE City IS NOT NULL;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

DistinctCities
3

A6. Count Distinct Product Categories

Objective

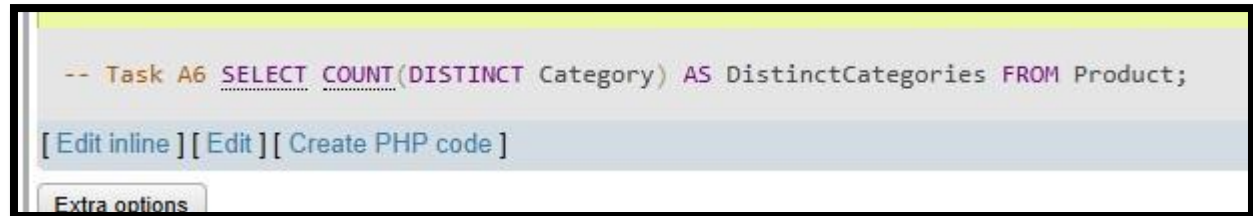
To find the number of unique product categories.

SQL Query

```
SELECT  
    COUNT(DISTINCT Category) AS DistinctCategories  
FROM Product;
```

Explanation

This query counts each unique product category only once.

A screenshot of a SQL query editor. The main text area contains the query: `-- Task A6 SELECT COUNT(DISTINCT Category) AS DistinctCategories FROM Product;`. Below the text area are three links: `[ Edit inline ]`, `[ Edit ]`, and `[ Create PHP code ]`. At the bottom left, there is a button labeled `Extra options`.

```
-- Task A6 SELECT COUNT(DISTINCT Category) AS DistinctCategories FROM Product;  
[ Edit inline ] [ Edit ] [ Create PHP code ]  
Extra options
```

A7. Customers With and Without City

Objective

To find customers who have a city and customers whose city is missing. ?

SQL Query

```
SELECT  
    COUNT(City) AS CustomersWithCity  
FROM Customer;  
  
SELECT  
    COUNT(*) - COUNT(City) AS CustomersWithoutCity  
FROM Customer;
```

Explanation

"COUNT(City)" counts non-NULL city values. The difference between total rows and nonNULL city values gives the number of customers without a city

```
-- Task A7 SELECT COUNT(City) AS CustomersWithCity FROM Customer;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

CustomersWithCity
7

```
SELECT COUNT(*) - COUNT(City) AS CustomersWithoutCity FROM Customer;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Extra options

CustomersWithoutCity
1

A8. Earliest and Latest Order Date

Objective

To find the first and latest order dates.

SQL Query

```
SELECT  
    MIN(OrderDate) AS EarliestOrderDate,  
    MAX(OrderDate) AS LatestOrderDate FROM  
OrderItem;
```

Explanation

"MIN()" finds the earliest date and "MAX()" finds the latest date

-- Task A8 <code>SELECT MIN(OrderDate) AS EarliestOrderDate, MAX(OrderDate) AS LatestOrderDate FROM OrderItem;</code> [Edit inline] [Edit] [Create PHP code]	
☐ Show all Number of rows: 25 ▼ Filter rows: Search this table	
Extra options	
EarliestOrderDate	LatestOrderDate
2023-03-10	2024-10-11

A9. Calculate Total Revenue

Objective

To calculate the total revenue generated from all orders.

SQL Query

```
SELECT
    SUM(oi.Quantity * p.Price) AS TotalRevenue
FROM OrderItem oi
JOIN Product p
    ON oi.ProdID = p.ProdID;
```

Explanation

The query joins the "OrderItem" and "Product" tables. Revenue is calculated by multiplying quantity by product price. "SUM()" calculates the total revenue

-- Task A9 <code>SELECT SUM(oi.Quantity * p.Price) AS TotalRevenue FROM OrderItem oi JOIN Product p ON oi.ProdID = p.ProdID;</code> [Edit inline] [Edit] [Create PHP code]	
☐ Show all Number of rows: 25 ▼ Filter rows: Search this table	
Extra options	
TotalRevenue	
476800.45	

A10. Calculate Average Order Quantity

Objective

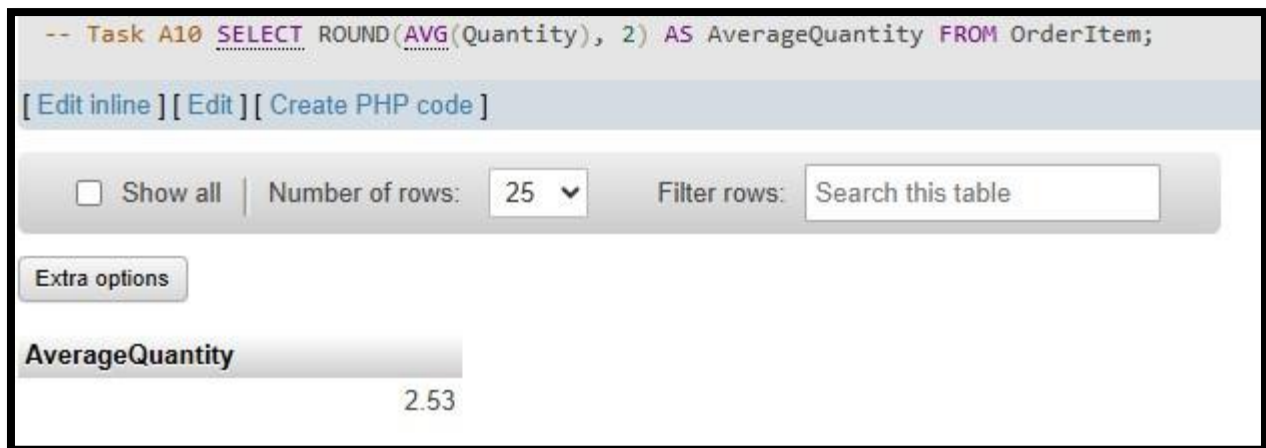
To find the average quantity of products ordered.

SQL Query

```
SELECT  
    ROUND(AVG(Quantity), 2) AS AverageQuantity  
FROM OrderItem;
```

Explanation

"AVG()" calculates the average quantity and "ROUND()" displays the result up to two decimal places.



PART B — GROUPING AND ADVANCED AGGREGATE FUNCTIONS

B1. Number of Customers by City

Objective

To count customers in each city.

SQL Query

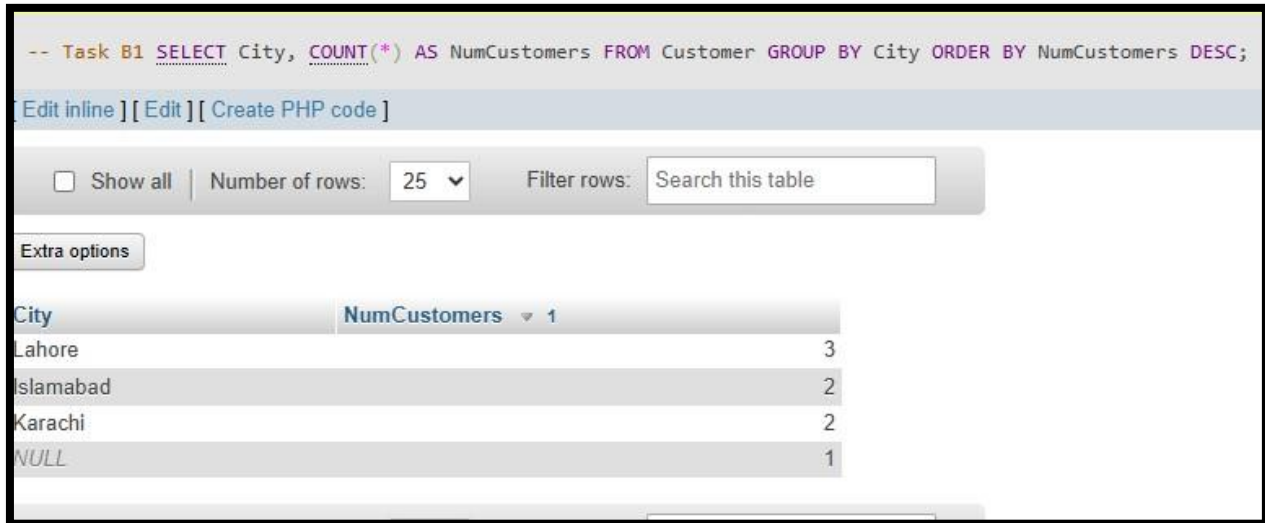
```
SELECT  
    City,  
    COUNT(*) AS NumCustomers  
FROM Customer
```

GROUP BY City

ORDER BY NumCustomers DESC;

Explanation

"GROUP BY City" creates a separate group for each city. "COUNT()" calculates the number of customers in each group.



The screenshot shows a SQL query execution interface. At the top, the query is: `-- Task B1 SELECT City, COUNT(*) AS NumCustomers FROM Customer GROUP BY City ORDER BY NumCustomers DESC;`. Below the query, there are links: [Edit inline](#), [\[Edit \]](#), and [\[Create PHP code \]](#). Below these links, there are controls: ☐ Show all, Number of rows: 25 (with a dropdown arrow), and Filter rows: Search this table (with a text input). Below these controls, there is a button labeled "Extra options". The results are displayed in a table with two columns: City and NumCustomers. The data is as follows:

City	NumCustomers
Lahore	3
Islamabad	2
Karachi	2
NULL	1

B2. Number of Products by Category

Objective

To count products in each category.

SQL Query

SELECT

Category,

COUNT(*) AS NumProducts

FROM Product

GROUP BY Category

ORDER BY NumProducts DESC;

Explanation

The query groups products by category and counts the products in each category.

```
-- Task B2 SELECT Category, COUNT(*) AS NumProducts FROM Product GROUP BY Category ORDER BY NumProducts DESC;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 ▼ | Filter rows:

Extra options

Category	NumProducts ▼ 1
Electronics	4
Stationery	2
Furniture	2
Grocery	2

☐ Show all | Number of rows: 25 ▼ | Filter rows:

B3. Average, Minimum and Maximum Price by Category

Objective

To calculate price statistics for every product category.

SQL Query

SELECT

Category,

ROUND(AVG(Price), 2) AS AvgPrice,

MIN(Price) AS MinPrice,

MAX(Price) AS MaxPrice

FROM Product

GROUP BY Category

ORDER BY AvgPrice DESC;

For every category, the query calculates the average, minimum, and maximum product price.

```
-- Task B3 SELECT Category, ROUND(AVG(Price), 2) AS AvgPrice, MIN(Price) AS MinPrice, MAX(Price) AS MaxPrice FROM Product GROUP BY Category ORDER BY AvgPrice DESC;
```

[Edit inline](#) | [Edit](#) | [Create PHP code](#)

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

	Category	AvgPrice ▼ 1	MinPrice	MaxPrice
☐ Edit Copy Delete	Electronics	48950.00		800.00
☐ Edit Copy Delete	Furniture	31750.25		18500.00
☐ Edit Copy Delete	Grocery	1275.00		650.00
☐ Edit Copy Delete	Stationery	400.00		350.00

☐ Check all With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows:

B4. Total Stock by Category

Objective

To calculate the total stock available in each category.

SQL Query

SELECT

Category,

SUM(StockQty) AS TotalStock

FROM Product

GROUP BY Category

ORDER BY TotalStock DESC;

Explanation

The query groups products by category and uses "SUM()" to calculate the total stock of each category.

-- Task B4 `SELECT Category, SUM(StockQty) AS TotalStock FROM Product GROUP BY Category ORDER BY TotalStock DESC;`

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

	Category	TotalStock
☐ Edit Copy Delete	Stationery	350
☐ Edit Copy Delete	Electronics	180
☐ Edit Copy Delete	Grocery	75
☐ Edit Copy Delete	Furniture	13

☐ Check all ☐ With selected Edit Copy Delete Export

B5. Number of Orders by Year

Objective

To count the number of orders placed in each year.

SQL Query

SELECT

YEAR(OrderDate) AS Year,

COUNT(*) AS NumOrders

FROM OrderItem

GROUP BY YEAR(OrderDate)

ORDER BY Year;

Explanation

"YEAR()" extracts the year from the order date. The orders are then grouped and counted by year;

-- Task B5
SELECT YEAR(OrderDate) AS Year, COUNT(*) AS NumOrders FROM OrderItem GROUP BY YEAR(OrderDate) ORDER BY Year;

[\[Edit inline \]](#)
[\[Edit \]](#)
[\[Create PHP code \]](#)

☐ Show all
Number of rows: 25
Filter rows: Search this table

Extra options

Year	NumOrders
2023	7
2024	8

B6. Number of Orders by Month in 2024

Objective

To count the orders made in each month of 2024.

SQL Query

SELECT

 MONTH(OrderDate) AS Month,

 COUNT(*) AS NumOrders

FROM OrderItem

WHERE YEAR(OrderDate) = 2024

GROUP BY MONTH(OrderDate)

ORDER BY Month;

Explanation

The "WHERE" condition selects only orders from 2024. "MONTH()" extracts the month and "COUNT()" counts the orders.

```
-- Task B6 SELECT MONTH(OrderDate) AS Month, COUNT(*) AS NumOrders FROM OrderItem WHERE YEAR(OrderDate) = 2024 GROUP BY MONTH(OrderDate) ORDER BY Month;
```

[Edit inline](#) [\[Edit \]](#) [\[Create PHP code \]](#)

☐ Show all | Number of rows: 25 ▼ | Filter rows:

Extra options

Month ▲ 1	NumOrders
1	2
2	1
4	2
7	1
9	1
10	1

B7. Categories with Average Price Greater Than 5000

Objective

To display categories whose average price is greater than 5000.

SQL Query

SELECT

Category,

ROUND(AVG(Price), 2) AS AvgPrice

FROM Product

GROUP BY Category

HAVING AVG(Price) > 5000;

Explanation

"GROUP BY" creates category groups, while "HAVING" filters those groups based on their average price.

```
-- Task B7 SELECT Category, ROUND(AVG(Price), 2) AS AvgPrice FROM Product GROUP BY Category HAVING AVG(Price) > 5000;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	Category	AvgPrice
☐ Edit Copy Delete	Electronics	48950.00
☐ Edit Copy Delete	Furniture	31750.25

↑ ☐ Check all With selected: Edit Copy Delete Export

B8. Cities Having More Than One Customer

Objective

To find cities that have more than one customer.

SQL Query

SELECT

City,

COUNT(*) AS NumCustomers

FROM Customer

WHERE City IS NOT NULL

GROUP BY City

HAVING COUNT(*) > 1;

Explanation

The query excludes "NULL" cities, groups customers by city, and displays only cities containing more than one customer

-- Task B8 `SELECT City, COUNT(*) AS NumCustomers FROM Customer WHERE City IS NOT NULL GROUP BY City HAVING COUNT(*) > 1;`

[Edit inline](#) [\[Edit \]](#) [\[Create PHP code \]](#)

☐ Show all | Number of rows: 25 ▼ | Filter rows:

Extra options

City	NumCustomers
Islamabad	2
Karachi	2
Lahore	3

☐ Show all | Number of rows: 25 ▼ | Filter rows:

B9 . Number of Orders per Customer

Objective

To count the number of orders for every customer, including customers who have no orders. ?

SQL Query

```
SELECT
    c.CustID,
    c.CustName,
    COUNT(oi.OrderID) AS NumOrders
FROM Customer c
LEFT JOIN OrderItem oi
    ON c.CustID = oi.CustID
GROUP BY c.CustID, c.CustName ORDER BY
    NumOrders DESC;
```

Explanation

A "LEFT JOIN" ensures that every customer is included. Customers with no orders receive a count of zero.

```
-- Task B9 SELECT c.CustID, c.CustName, COUNT(oi.OrderID) AS NumOrders FROM Customer c LEFT JOIN OrderItem oi ON c.CustID = oi.CustID GROUP BY c.CustID, c.CustName ORDER BY NumOrders DESC;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

CustID	CustName	NumOrders
2	Sara Iqbal	3
3	Hamza Raza	3
7	Usman Tariq	3
1	Ali Khan	2
5	Bilal Ahmed	2
4	Ayesha Noor	1
6	Fatima Sheikh	1
8	Maira Javed	0

B10. Total Quantity Sold per Product

Objective

To calculate the total quantity sold for every product, including products that were never sold.

SQL Query

```
SELECT
    p.ProdName,
    COALESCE(SUM(oi.Quantity), 0) AS TotalQty
FROM Product p
LEFT JOIN OrderItem oi
    ON p.ProdID = oi.ProdID
GROUP BY p.ProdID, p.ProdName
ORDER BY TotalQty DESC;
```

Explanation

The "LEFT JOIN" includes all products. "COALESCE()" changes "NULL" into zero for products that have no sales.

```
-- Task B10 SELECT p.ProdName, COALESCE(SUM(oi.Quantity), 0) AS TotalQty FROM Product p LEFT JOIN OrderItem oi ON p.ProdID = oi.ProdID GROUP BY p.ProdID, p.ProdName ORDER BY TotalQty DESC;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

ProdName	TotalQty
Notebook A4	15
Coffee Beans 1kg	5
USB-C Cable	4
Wireless Mouse	3
Green Tea Box	3
Laptop Pro 15	2
Ballpoint Pen 10pk	2
Bluetooth Speaker	2
Office Chair	1
Standing Desk	1

B11 . Revenue by Product Category

Objective

To calculate the total revenue generated by each product category.

SQL Query

SELECT

p.Category,

SUM(oi.Quantity * p.Price) AS Revenue

FROM OrderItem oi

JOIN Product p

ON oi.ProdID = p.ProdID

GROUP BY p.Category

ORDER BY Revenue DESC;

Explanation

The query calculates revenue using "Quantity × Price" and groups the revenue according to product category.

```
-- Task B11 SELECT p.Category, SUM(oi.Quantity * p.Price) AS Revenue FROM OrderItem oi JOIN Product p ON oi.ProdID = p.ProdID GROUP BY p.Category
ORDER BY Revenue DESC;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

Category	Revenue
Electronics	395700.00
Furniture	63500.50
Grocery	11449.95
Stationery	6150.00

☐ Show all | Number of rows: 25 | Filter rows: Search this table

B12 . Total Spending per Customer

Objective

To calculate the total amount spent by each customer, including customers with no orders.

? SQL Query

```
SELECT
    c.CustName,
    COALESCE(SUM(oi.Quantity * p.Price), 0) AS TotalSpend
FROM Customer c
LEFT JOIN OrderItem oi
    ON c.CustID = oi.CustID
LEFT JOIN Product p
    ON oi.ProdID = p.ProdID
GROUP BY c.CustID, c.CustName
ORDER BY TotalSpend DESC;
```

Explanation

The query calculates the total spending of each customer. "LEFT JOIN" includes customers without orders, while "COALESCE()" displays their spending as zero.

Showing rows 1 - 7 (Total: 8 rows, took 0.0017 seconds)

```
-- Task B12 SELECT c.CustName, COALESCE(SUM(oi.Quantity * p.Price), 0) AS TotalSpend FROM Customer c LEFT JOIN OrderItem oi ON c.CustID = oi.CustID
LEFT JOIN Product p ON oi.ProdID = p.ProdID GROUP BY c.CustID, c.CustName ORDER BY TotalSpend DESC;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

CustName	TotalSpend
Hamza Raza	196299.98
Ali Khan	190000.00
Fatima Sheikh	45000.50
Sara Iqbal	27750.00
Usman Tariq	6350.00
Bilal Ahmed	5700.00
Ayesha Noor	5699.97
Maira Javed	0.00

Activate Windows
Go to Settings to activate Windows.

B13. Customers Spending More Than 50,000

Objective

To find customers whose total spending is greater than 50,000.

SQL Query

SELECT

c.CustName,

SUM(oi.Quantity * p.Price) AS TotalSpend

FROM Customer c

JOIN OrderItem oi

ON c.CustID = oi.CustID

JOIN Product p

ON oi.ProdID = p.ProdID

GROUP BY c.CustID, c.CustName

HAVING SUM(oi.Quantity * p.Price) > 50000;

Explanation

The query calculates total spending for each customer and uses "HAVING" to display customers whose spending exceeds 50,000.

```
-- Task B13 SELECT c.CustName, SUM(oi.Quantity * p.Price) AS TotalSpend FROM Customer c JOIN OrderItem oi ON c.CustID = oi.CustID JOIN Product p ON oi.ProdID = p.ProdID GROUP BY c.CustID, c.CustName HAVING SUM(oi.Quantity * p.Price) > 50000;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

CustName	TotalSpend
Ali Khan	190000.00
Hamza Raza	196299.98

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

B14. Customer Count and Revenue by City

Objective

To display the number of customers and total revenue for cities having more than one customer.

SQL Query

```
SELECT  c.City,
        COUNT(DISTINCT c.CustID) AS NumCustomers,
        COALESCE(SUM(oi.Quantity * p.Price), 0) AS TotalRevenue
FROM Customer c
LEFT JOIN OrderItem oi
    ON c.CustID = oi.CustID
LEFT JOIN Product p
    ON oi.ProdID = p.ProdID
WHERE c.City IS NOT NULL
GROUP BY c.City
HAVING COUNT(DISTINCT c.CustID) > 1;
```

Explanation

The query groups customers by city and calculates customer count and total revenue.

"COUNT(DISTINCT CustID)" prevents a customer from being counted multiple times.

```
-- Task B14 SELECT c.City, COUNT(DISTINCT c.CustID) AS NumCustomers, COALESCE(SUM(oi.Quantity * p.Price), 0) AS TotalRevenue FROM Customer c LEFT JOIN OrderItem oi ON c.CustID = oi.CustID LEFT JOIN Product p ON oi.ProdID = p.ProdID WHERE c.City IS NOT NULL GROUP BY c.City HAVING COUNT(DISTINCT c.CustID) > 1;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

City	NumCustomers	TotalRevenue
Islamabad	2	5699.97
Karachi	2	33450.00
Lahore	3	392649.98

B15 . Top 3 Best . Selling Products

Objective

To find the three products with the highest quantity sold.

? SQL Query

SELECT

p.ProdName,

SUM(oi.Quantity) AS TotalQty

FROM Product p

JOIN OrderItem oi

ON p.ProdID = oi.ProdID

GROUP BY p.ProdID, p.ProdName

ORDER BY TotalQty DESC LIMIT 3;

Explanation

The query calculates the total quantity sold for each product, sorts the results from highest to lowest, and displays the top three products.

```
-- Task B15 SELECT p.ProdName, SUM(oi.Quantity) AS TotalQty FROM Product p JOIN OrderItem oi ON p.ProdID = oi.ProdID GROUP BY p.ProdID, p.ProdName
ORDER BY TotalQty DESC LIMIT 3;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

ProdName	TotalQty ▾ 1
Notebook A4	15
Coffee Beans 1kg	5
USB-C Cable	4

B16. Revenue by Year

Objective

To calculate the total revenue generated in each year.

SQL Query

SELECT

YEAR(oi.OrderDate) AS Year,

SUM(oi.Quantity * p.Price) AS Revenue

FROM OrderItem oi

JOIN Product p

ON oi.ProdID = p.ProdID

GROUP BY YEAR(oi.OrderDate)

ORDER BY Year;

Explanation

The query calculates revenue using quantity and price and groups the revenue according to the year of the order.

```
-- Task B16 SELECT YEAR(oi.OrderDate) AS Year, SUM(oi.Quantity * p.Price) AS Revenue FROM OrderItem oi JOIN Product p ON oi.ProdID = p.ProdID GROUP
BY YEAR(oi.OrderDate) ORDER BY Year;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 ▾ | Filter rows:

Extra options

Year ▴ 1	Revenue
2023	408449.97
2024	68350.48

B17. Average Order Value

Objective

To calculate the average value of all orders.

SQL Query

```
SELECT
    ROUND(AVG(OrderValue), 2) AS AverageOrderValue
FROM (
    SELECT
        OrderID,
        SUM(Quantity * p.Price) AS OrderValue
    FROM OrderItem oi
    JOIN Product p
        ON oi.ProdID = p.ProdID
    GROUP BY OrderID )
AS OrderTotals;
```

Explanation

The inner query calculates the total value of each order. The outer query calculates the average of all order values and rounds the result to two decimal places.

6. Important Aggregate Functions Used

Function | Purpose

"**COUNT()**" | Counts rows

"**SUM()**" | Calculates total

"**AVG()**" | Calculates average

"**MIN()**" | Finds minimum value

"**MAX()**" | Finds maximum value

"**DISTINCT**" | Removes duplicate values

"**GROUP BY**" | Groups similar records

"**HAVING**" | Filters groups

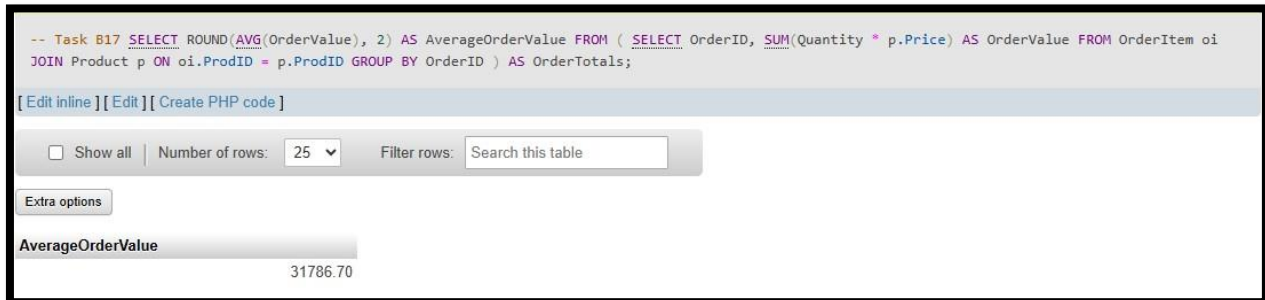
"**COALESCE()**" | Replaces NULL with another value

"**ROUND()**" | Rounds a numeric value

"**YEAR()**" | Extracts year from a date "**MONTH()**" |

Extracts month from a date

.



```
-- Task B17 SELECT ROUND(AVG(OrderValue), 2) AS AverageOrderValue FROM ( SELECT OrderID, SUM(Quantity * p.Price) AS OrderValue FROM OrderItem oi
JOIN Product p ON oi.ProdID = p.ProdID GROUP BY OrderID ) AS OrderTotals;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

AverageOrderValue
31786.70

7. Conclusion

In this lab, aggregate functions in SQL were successfully practiced using a RetailStore database. The "COUNT()", "SUM()", "AVG()", "MIN()", and "MAX()" functions were used to perform calculations on database records. "GROUP BY" was used to organize data into meaningful groups, while "HAVING" was used to filter grouped results. The lab also demonstrated the use of "JOIN", "LEFT JOIN", "DISTINCT", "COALESCE()", and date functions with aggregate queries. These techniques are useful for generating summaries and extracting meaningful information from large databases.

*****THE END*****